

SynapseTutor AI — Architecture, Techniques & Engineering Report

Executive Summary: Production-grade Multimodal Adaptive Tutoring Platform combining Multimodal Source-Grounded RAG, Bayesian Knowledge Tracing (BKT), Scikit-Learn Student Readiness Models, Three-Tier Cognitive Memory, Self-Verifying Assessments, and Local GPU LLM Inference (Ollama Qwen 2.5 7B on CUDA).

1. Four-Layer System Architecture

- **Layer 1 — Interactive Frontend (React 19 + TypeScript + Vite + Tailwind CSS + KaTeX):** Includes 5 core modules: *Dashboard*, *Tutor Chat* (multi-session RAG with KaTeX math & Mermaid diagrams), *Adaptive Tests*, *Course Library*, and *Learner Profile*, plus the slide-over *Source Viewer Drawer*.
- **Layer 2 — FastAPI Cognitive Orchestration Layer:** Async REST APIs (`/api/v1`) coordinating the Multimodal Ingestion Worker, Hybrid Grounded RAG Engine, Learner Cognitive Engine, 3-Tier Memory System, and Self-Verifying Assessment Pipeline.
- **Layer 3 — AI & ML Compute Layer:** Powered by Local Ollama `qwen2.5:7b` (NVIDIA RTX 3050 6GB CUDA GPU), `sentence-transformers/all-MiniLM-L6-v2` dense embeddings, `FlashRank` cross-encoder reranking, `scikit-learn` (Random Forest & Gradient Boosting), and `SymPy` deterministic math verification, with automatic fallback to `MockLLMProvider`.
- **Layer 4 — Hybrid Persistence Layer:** SQLite (WAL mode + Async SQLAlchemy 2.0) for relational state and ChromaDB for source-anchored dense vector chunks.

2. Core AI & Machine Learning Techniques

Technique 1: Multimodal Course Ingestion & Canonical Source Anchoring

Ingests PDF textbooks (`PyMuPDF` for text, tables, equations, figures), PowerPoint slide decks (`python-pptx` for slides & speaker notes), and lecture videos (`yt-dlp` & `faster-whisper` for timestamped transcripts). Every vector chunk retains strict provenance metadata (e.g., `[Source: Module 1 • Slide 24]`) linked directly to the Source Viewer Drawer.

Technique 2: Hybrid Grounded RAG + Cross-Encoder Reranking + Off-Topic Refusal

Rewrites follow-up queries using short-term dialogue context, then executes hybrid retrieval combining Dense Vector Search (ChromaDB) and BM25 Lexical Keyword Scoring, followed by `FlashRank` cross-encoder reranking. Strictly refuses out-of-syllabus queries while offering an opt-in *Explain with Outside Knowledge* toggle.

Technique 3: Bayesian Knowledge Tracing (BKT) & Scikit-Learn Student Models

Tracks per-concept mastery probability $P(L_t)$ initialized at $P(L_0) = 0.15$ for novices:

$$P(L_t \mid \text{Correct}) = [P(L_t) * (1 - P(S))] / [P(L_t) * (1 - P(S)) + (1 - P(L_t)) * P(G)]$$
$$P(L_{t+1}) = P(L_t \mid \text{Obs}) + (1 - P(L_t \mid \text{Obs})) * P(T)$$

Incorporates evidence weighting (Quiz = 1.0, Short Answer = 0.7, Chat = 0.3) and Ebbinghaus forgetting decay. Pairs with a **Random Forest Classifier** (predicting `ready_for_next`, `needs_practice`, or `prerequisite_gap`) and a **Gradient Boosting Regressor** (predicting expected score and study minutes to reach 80% mastery).

Technique 4: Three-Tier Cognitive Memory Architecture

- **Short-Term Memory:** Sliding turn window for active chat session coherence.
- **Long-Term Memory:** Persistent SQLite mastery records + adaptive Learning Behavior Profile (automatically switches to `intuitive_analogy_first` when student confusion is detected).
- **Episodic Memory:** Timestamped timeline of struggles, misconceptions, and breakthroughs injected into future tutor prompts.

Technique 5: Self-Verifying Adaptive Assessment Engine

Samples questions via a 50% weak / 30% intermediate / 20% review split (plus Cold-Start Diagnostic mode). Every generated question passes through a deterministic **SymPy symbolic verifier**, distractor validator, and dense cosine similarity novelty check (>0.85 rejected for 0% question repetition).

3. Complete Engineering Log — What Was Configured & Fixed

#	Component / Task	Issue / Requirement	Engineering Solution Implemented
1	Full-Stack Environment & Server Startup	Run FastAPI backend and React/Vite frontend concurrently on Windows.	Verified Python 3.13 & Node.js v24, audited 179 npm packages, initialized clean SQLite WAL schema, and launched FastAPI (:8000) and Vite (:5173).
2	Resilient LLM & Vision Auto-Fallback Fix	Tutor Chat showed <code>Error: Failed to fetch</code> (HTTP 500) when <code>.env</code> pointed to an offline Ollama instance without exception handling.	Added resilient <code>try/except</code> fallback logic in <code>OllamaLLMProvider</code> , <code>GeminiLLMProvider</code> , and <code>OllamaVisionProvider</code> to fall back gracefully to <code>MockLLMProvider</code> without crashing.
3	Local Ollama 7B (<code>qwen2.5:7b</code>) GPU Integration	Connect the platform to a local 7B model with GPU acceleration.	Started <code>ollama serve</code> on <code>127.0.0.1:11434</code> using the NVIDIA GeForce RTX 3050 6GB GPU (CUDA) , configured <code>LLM_PROVIDER=ollama</code> & <code>OLLAMA_MODEL=qwen2.5:7b</code> , and verified live RAG inference.
4	KaTeX Mathematical Formula Rendering Fix	<code>qwen2.5:7b</code> outputs LaTeX delimiters <code>\(...\)</code> and <code>\[...\]</code> which were displayed as raw unformatted text in chat.	Upgraded the inline tokenizer regex and multiline block parser in <code>FormattedMessage.tsx</code> to parse <code>\(...\)</code> , <code>\[...\]</code> , <code>\$...\$</code> , and <code>...</code> and render them via KaTeX.
5	UI Navigation Streamlining	Remove Knowledge Explorer and Analytics tabs to focus the UI.	Updated <code>Navbar.tsx</code> and <code>App.tsx</code> to remove Knowledge Explorer and Analytics, leaving 5 focused modules: Dashboard, Tutor Chat, Adaptive Tests, Course Library, and Learner Profile.